

아무도 알려주지 않았던

신입 개발자의 실무 이야기

2년차 백엔드 개발자 **장우성**

"약 1년동안 현업에서 일하면서 느낀 가벼운 인사이트를 경험과 함께 공유하고자 합니다."

목차

01. 연사자 소개

02. 협업 마인드

기술력보다 먼저 갖춰야할 '태도'

03. 실무에 필요한 지식

우리가 이론적으로 배웠던 CS 어디에 필요한걸까?

04. 커리어 준비

방황하는 1년차, 커리어 관리는 어떻게?

연사자 소개

누구신지..?

연사자 소개

HPC 업계에 종사하는 만 2년차 백엔드 개발자

장우성

- 명지대학교 컴퓨터공학과 졸업
- 다원컴퓨팅(주) 백엔드 개발자로 재직 중

Backend Engineer

HPC 도메인

Observability



What is HPC?

고성능 컴퓨팅 (High-Performance Computing)

단일 컴퓨터가 처리할 수 없는 거대하고 복잡한 계산을 수백 ~ 수천 대의 서버를 연결하여 초고속으로 처리하는 기술입니다.

데이터센터 모니터링 플랫폼 개발

HPC 업계에 종사하는 백엔드 개발자로서
HPC 데이터센를 통합적으로 모니터링하여 장애를 빠르게 감지하고
효율적으로 운영할 수 있도록 합니다.

협업 마인드

사내 빌런 에피소드

01 실제 목격한 '팀을 망치는 사람들'

- 입사 초기, 1년 차·5년 차 선배 두 명이 함께 다니며 동료들에 대한 불만과 비판을 자주 표출함
- 같은 팀 동료들의 부족한 점을 지속적으로 지적하며 팀 내 갈등 발생
- 프로젝트 진행 중 마찰을 빚은 후 두 명 모두 갑작스럽게 중도 퇴사
- 퇴사로 인해 발생한 업무 공백과 미완료 업무를 인수받아 수습해야 했음

- **정말 많은 시간을 같이 보낸다:** 하루 중 가장 긴 시간을 동료와 함께합니다.
- **단힌 마음은 모두를 괴롭게 한다:** 내가 상대가 미운 만큼 상대도 나를 힘들어합니다. 감정 소모는 프로젝트의 독입니다.
- **환경을 부정하지 말고 받아들이자:** 현재의 환경에 마음에 들지 않더라도 피하지 말고 맞서자

실제 사내 에피소드: 팀을 망치는 사람들



02 공유 환경을 만들자

서로 성향이 맞지 않고 직접적인 소통이 어렵다면 공유 시스템을 통해 **소통 비용을 최소화**하자



Swagger

API 스펙 문서를 자동화하여 프론트엔드와 백엔드 간 오해와 감정 소모를 원천 차단



Notion

의사결정 맥락과 기록을 투명하게 남깁니다.
"그때 그렇게 말하셨잖아요" 식의 소모적 논쟁을 방지



GitLab / GitHub

코드를 사람이 아닌 '시스템' 차원에서 함께 검증하는 절차를 강제하여 감정이 상하지 않는 건강한 피드백 문화를 구축합니다.

열린 마인드로 업무를 잘 공유하고 소통하는 **히어로**가 되자

02

실무 지식

미리 알아두면 좋을만한

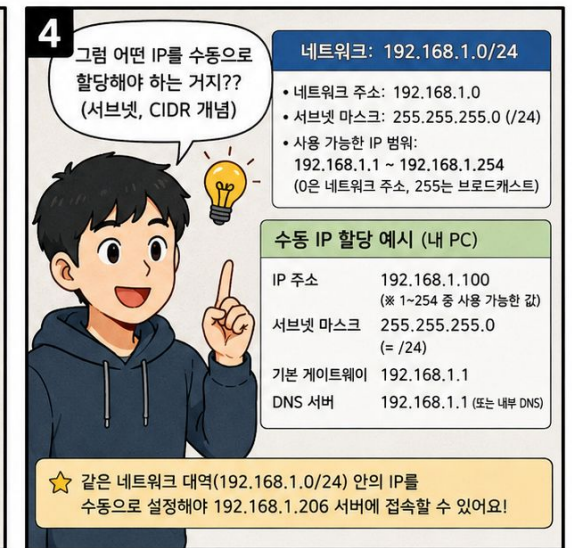
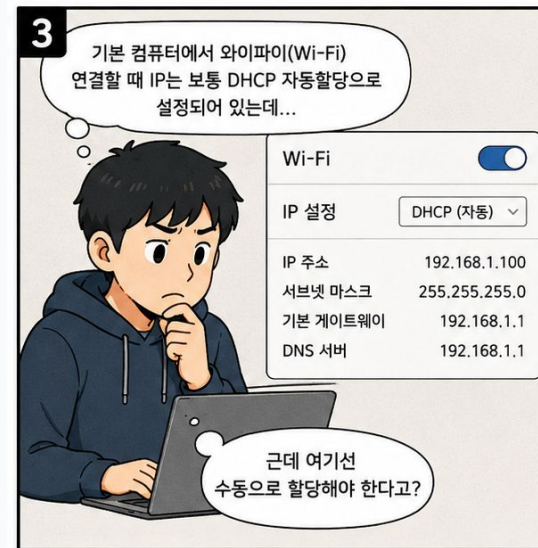
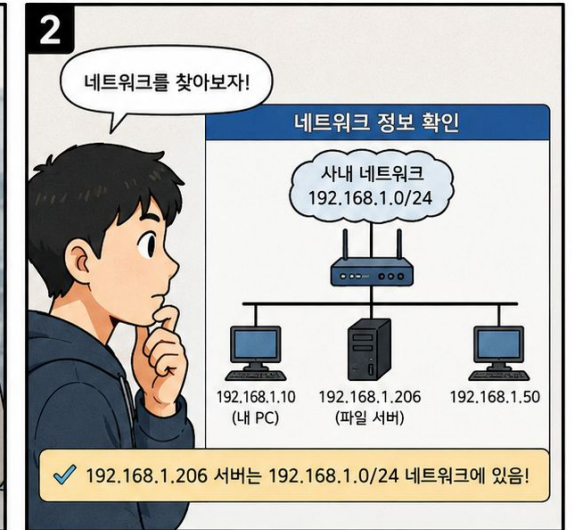
기술적인 지식들

01 네트워크

사내망 접속하기

- "192.168.1.206 서버에 접속해서 파일 확인해보세요"라는 요청을 받음
- 어떤 네트워크에 연결해야 하는지 확인
- 내 PC의 네트워크 설정 변경
- 서버에 정상 접속

- **IP 주소:** 네트워크에서 장비를 식별하는 고유 번호
- **서브넷 마스크:** IPv4주소 체계에서 같은 대역을 구분하는 기준
- **DHCP:** IP를 자동으로 받는 방식, 수동 설정 필요 여부 판단 기준
- **게이트웨이:** 외부로 나가는 출구 IP, 다른 대역 통신 시 필요
- **DNS 서버:** 도메인 <-> IP 매핑 및 변환



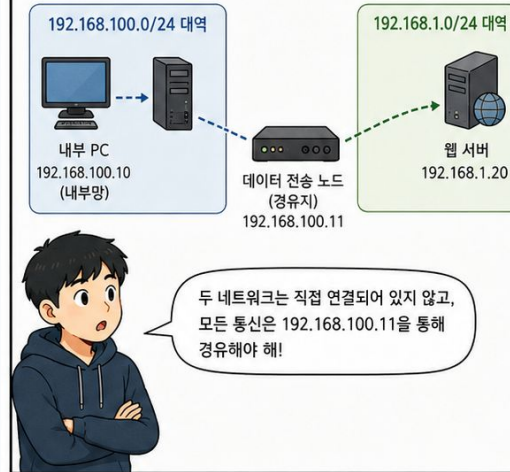
01 네트워크

다른 네트워크 서버 접속

- 내부 서버는 접속되는데 대상 서버는 접속되지 않음
- 중간에 거쳐 가는 서버(데이터 전송 노드) 존재 여부 확인
- 내 PC에도 접속 경로 설정 추가
- 그래도 안 되면 중간 서버의 통신 정책 점검

- **서브넷 / 대역 분리:** 두 네트워크가 다른 대역이면 직접 통신 불가, 경유지 필요
- **게이트웨이/ 라우터:** 다른 대역으로 패킷을 전달해주는 경유지
- **라우팅 테이블:** 목적지 IP에 따라 어디로 보낼지 결정하는 지도
- **0.0.0.0:** 라우팅 테이블에 없을 시에 전달하는 기본 경로
- **iptables:** 리눅스 패킷 필터링 도구
- **FORWARD 체인:** 경유지를 통과하는 패킷에 적용되는 규칙

1 두 개의 네트워크 대역과 데이터 전송 노드(경유지)



2 내부 서버의 라우팅 규칙

내부 서버(192.168.100.10)에서는 192.168.1.0/24 대역으로 가는 모든 패킷을 192.168.100.11(전송 노드)로 보내도록 라우팅 규칙이 설정되어 있어!

목적지 네트워크	넷마스크	게이트웨이
192.168.100.0	255.255.255.0	직접 연결
192.168.1.0	255.255.255.0	192.168.100.11
0.0.0.0	0.0.0.0	기본 게이트웨이

★ 즉, 내부망에서 192.168.1.0/24 대역으로 통신할 때는 반드시 192.168.100.11을 거쳐 간다!

3 로컬 PC에서 웹 서버(192.168.1.20)에 직접 접속하려면?

내 로컬 PC는 기본적으로 192.168.1.0/24 대역을 직접 연결로 인식하지 못하니까, 라우팅을 직접 설정해줘야 해!

직접 접속 시도

내 로컬 PC

라우팅 설정 후

웹 서버
192.168.1.20

192.168.100.11
(데이터 전송 노드)

Windows (명령 프롬프트)

```
route add 192.168.1.0 mask 255.255.255.0 192.168.100.11 -p
→ 영구 저장(-p) 옵션
```

Linux (터미널)

```
sudo ip route add 192.168.1.0/24 via 192.168.100.11
→ 재부팅 후도 유지하려면 /etc/network/interfaces
또는 라우팅 스크립트에 추가
```

✓ 설정 후, 로컬 PC에서도 192.168.1.20 웹 서버에 접속 가능!

4 라우팅이 설정됐는데도 안 된다면? iptables 확인!

라우팅도 했는데 접속이 안 돼? 그럼 데이터 전송 노드의 iptables(패킷 정책) 설정을 확인해보자!

체크 포인트

- 포워딩(Forwarding) 허용 여부
- 특정 포트(80, 443 등) 차단 여부
- 출발지/목적지 IP 제한 여부

iptables가 차단하면 라우팅이 있어도 패킷은 통과 불가!

해결 예시

```
sudo iptables -A FORWARD -s 192.168.100.0/24 -d 192.168.1.0/24 -j ACCEPT
sudo iptables -save > /etc/iptables/rules.v4
(또는 서버스에 저장)
```

iptables 상태 확인 예시

```
sudo iptables -L -n -v
```

Chain	Policy	Target	Source	Destination
Chain FORWARD (policy ACCEPT)				
iptables	ACCEPT	all	192.168.100.0/24	192.168.1.0/24
0	DROP	all	0.0.0.0/0	192.168.1.20

👍 라우팅 설정 + iptables 정책 허용 → 비로소 통신 성공!

01 네트워크

다른 네트워크 서버 접속

- 내부 서버는 접속되는데 대상 서버는 접속되지 않음
- 중간에 거쳐 가는 서버(데이터 전송 노드) 존재 여부 확인
- 내 PC에도 접속 경로 설정 추가
- 그래도 안 되면 중간 서버의 통신 정책 점검

- **서브넷 / 대역 분리:** 두 네트워크가 다른 대역이면 직접 통신 불가, 경유지 필요
- **게이트웨이/ 라우터:** 다른 대역으로 패킷을 전달해주는 경유지
- **라우팅 테이블:** 목적지 IP에 따라 어디로 보낼지 결정하는 지도
- **0.0.0.0:** 라우팅 테이블에 없을 시에 전달하는 기본 경로
- **iptables:** 리눅스 패킷 필터링 도구
- **FORWARD 체인:** 경유지를 통과하는 패킷에 적용되는 규칙

학교에서 배운 네트워크 지식은 정말 중요하다!

1 두 개의 네트워크 대역과 데이터 전송 노드(경유지)

192.168.100.0/24 대역

내부 PC
192.168.100.10
(내부망)

데이터 전송 노드
(경유지)
192.168.100.11

192.168.1.0/24 대역

웹 서버
192.168.1.20

두 네트워크는 직접 연결되어 있지 않고, 모든 통신은 192.168.100.11을 통해 경유해야 해!

2 내부 서버의 라우팅 규칙

내부 서버(192.168.100.10)에서는 192.168.1.0/24 대역으로 가는 모든 패킷을 192.168.100.11(전송 노드)로 보내도록 라우팅 규칙이 설정되어 있어!

목적지 네트워크	넷마스크	게이트웨이
192.168.100.0	255.255.255.0	직접 연결
192.168.1.0	255.255.255.0	192.168.100.11
0.0.0.0	0.0.0.0	기본 게이트웨이

★ 즉, 내부망에서 192.168.1.0/24 대역으로 통신할 때는 반드시 192.168.100.11을 거쳐 간다!

3 로컬 PC에서 웹 서버(192.168.1.20)에 직접 접속하려면?

내 로컬 PC는 기본적으로 192.168.1.0/24 대역을 직접 연결로 인식하지 못하니까, 라우팅을 직접 설정해줘야 해!

Windows (영양 프로그램)

```
route add 192.168.1.0 mask 255.255.255.0 192.168.100.11 -p
→ 영구 저장(-p) 옵션
```

Linux (터미널)

```
sudo ip route add 192.168.1.0/24 via 192.168.100.11
→ 재부팅 후도 유지하려면 /etc/network/interfaces
또는 라우팅 스크립트에 추가
```

4 라우팅이 설정됐는데도 안 된다면? iptables 확인!

라우팅도 했는데 접속이 안 돼? 그럼 데이터 전송 노드의 iptables(패킷 정책) 설정을 확인해보자!

체크 포인트

- 포워딩(Forwarding) 허용 여부
- 특정 포트(80, 443 등) 차단 여부
- 출발지/목적지 IP 제한 여부

iptables가 차단하면 라우팅이 있어도 패킷은 통과 불가!

해결 예시

```
sudo iptables -A FORWARD -s 192.168.100.0/24 -d 192.168.1.0/24 -j ACCEPT
```

```
sudo iptables -L -n -v
```

Chain	Policy	Filter	Bytes	Target	Prot	Opt	In	Out	Source	Destination
Chain FORWARD	(policy ACCEPT)									

sudo iptables-save > /etc/iptables/rules.v4 (또는 서비스에 저장)

02 DB

아찔했던 경험

HPC 환경에서 메트릭 데이터가 수천만 건 쌓인 상황.
쿼리 하나 잘못 짰다가 **풀스캔(Full Scan)** 발생 → 해당 기능을 수행하는데 1분이 걸리는 성능 장애 경험.

- **풀스캔 vs 인덱스:** EXPLAIN으로 쿼리 실행계획을 확인하는 습관이 필수입니다. 인덱스를 제대로 타는지 반드시 확인하세요.
- **설계 단계가 제일 중요:** DB는 한번 만들면 스키마 변경이 매우 어렵습니다. 나중에 바꾸는 비용이 처음에 잘 만드는 비용보다 압도적으로 큼니다.
- **성능 모니터링 습관:** 슬로우 쿼리 로그 모니터링. 인덱스 최적화 후 쿼리 실행시간이 **2.3초 → 0.3ms**로 줄어드는 것을 직접 경험해 보세요.

<https://wu-seong.github.io/posts/postgresql-slow-query-7000x/>

조회 쿼리의 경우 **풀스캔** 가능성을 고려하고 조회 성능을 모니터링하자



Slow Query

7000배 개선하기

실행 계획 분석과
LATERAL JOIN 활용

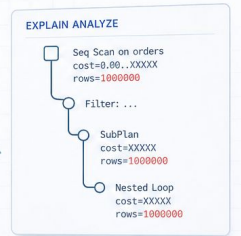
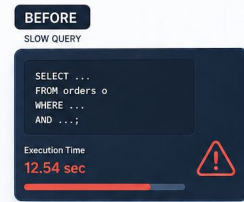


실행 계획 분석
EXPLAIN (ANALYZE)

병목 원인 파악
비효율적 서브쿼리

LATERAL JOIN 적용
필요한 데이터만 조회

7000배 성능 개선
쿼리 시간 대폭 단축



03 운영 환경에서 Logging의 중요성

로그 없는 운영은 눈 감고 운전하기

로컬에선 완벽했던 코드가 운영 서버에만 올라가면 제대로 동작이 안되는 경우가 있습니다. 로깅이 안 되어 있다면 원인을 알기 위해 코드 수정 → 빌드 → 배포라는 고통스러운 루프를 반복해야 합니다.

Panel 1: 메트릭 수집 장애 발생!
하지만 로그 파일은 깨끗...

Panel 2: 로깅 코드 추가 후
N분간의 지루한 빌드 대기

Panel 3: 배포 완료. "아, 레벨
설정을 잘못했네..." 다시 빌드 시작

Panel 4: 처음부터 로그 레벨을
설계했다면 어땠을까?



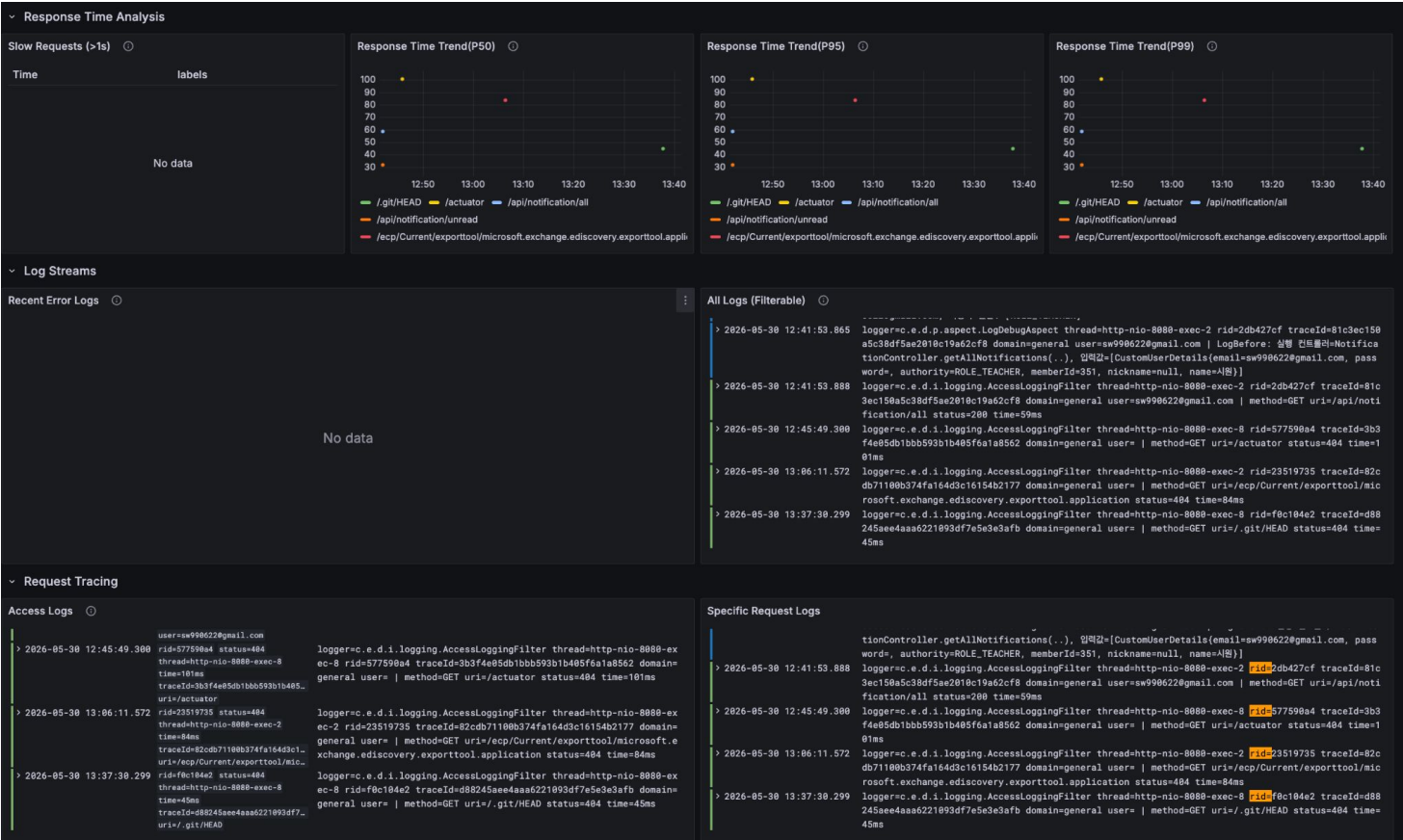
03 운영 환경에서 Logging의 중요성

터미널을 벗어난 통합 대시보드

서버에 일일이 접속하여 tail -f를 치는 시대는 지났습니다.
Loki와 Grafana를 연동하면 단일 대시보드에서 다수의
컨테이너 로그를 실시간으로 탐색할 수 있습니다.

Grafana: 강력한 쿼리(LogQL) 기반 시각화 및 알림

Loki: 인덱싱을 최소화한 고효율 로그 저장소



원활한 운영을 위해 기본적인 **운영 모니터링 환경**을 구축하자

03 운영 환경에서 Logging의 중요성

관측 가능성 (Observability)으로의 진화



Metrics

시스템의 건강 상태를 수치로 표현 (CPU, Memory, 응답 속도).
'무엇'이 문제인지 빠르게 감지합니다.



Logs

특정 시점에 발생한 이벤트의 상세 기록. '왜' 발생했는지 맥락을 파악하는 핵심 도구입니다.



Tracing

요청이 여러 서비스를 거치는 흐름을 추적. 분산 환경에서 '어디서' 지연이 생기는지 찾아냅니다.

심화 학습 키워드

#OpenTelemetry #Prometheus #Jaeger #SLI/SLO #Distributed_Tracing

"데이터를 많이 수집하는 것보다, 어떤 유의미한 신호를 볼지 설계하는 것이 더 중요합니다."

03

커리어 관리

신입 개발자로 살아남기

01 이직준비 = 커리어 관리

1. 아닌 곳은 1개월 안에 판단

연봉, 복지, 워라벨, 사람 등을 종합적으로 판단해서 1개월 안에 결단을 내리시는 걸 추천드립니다.

2. 면접은 경험이 쌓인 후에

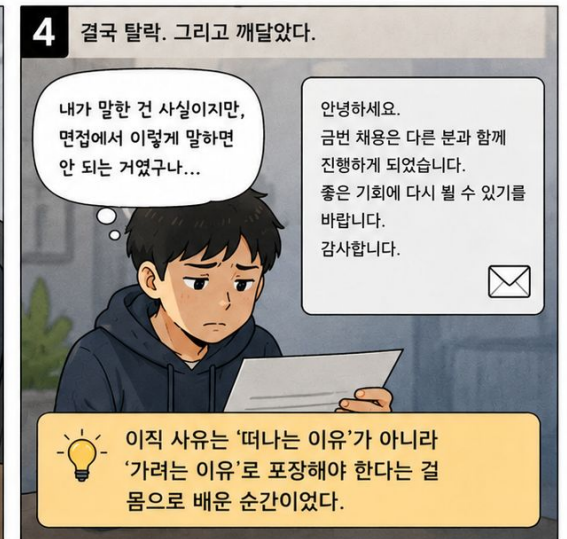
최소 6개월~1년 치 성과가 있어야 면접관 앞에서 할 말이 생깁니다. 충분한 준비를 통해 무기를 갖추시다.

3. 하지만 '이직무새'는 안 된다

의미 있는 문제 해결 경험이 결국 나의 경쟁력입니다. 불만만 갖기보다 한번 제대로 몰입해서 성과를 내야 합니다.

4. 이직 준비는 항상 해야 한다

평소에 문서화를 잘 해두고(가능하면 수치화) 이력서와 포트폴리오는 평소에 분기별로 한번씩은 최신 상태로 업데이트 해두어야 기회를 잡습니다.



02 사이드 프로젝트는 신중하게

장점

- 실무에서 못 해보는 기술적 갈증을 채울 수 있습니다.
- 프로젝트 선정 시 기획/PM의 역량이 매우 중요합니다.
- 같이 하는 사람 자체가 소중한 **네트워킹 자산**이 됩니다.

단점

- 퇴근을 하고도 일을 하는 듯한 느낌..
- 운영 경험을 목적으로 하는 경우, 확정적으로 오래걸릴 수 있다.
- 팀 프로젝트의 경우 여기서도 동일하게 사람과 함께 하기 때문에 또 다른 스트레스를 받을 수 있다.

1 운영 경험, 협업 경험, 성장 등을 얻고 싶어서 사이드 프로젝트에 합류했다.

이 프로젝트에서 운영 경험도 쌓고, 협업도 배우면서 많이 성장하고 싶어요!

우리의 목표

- ☒ 사용자에게 가치 있는 서비스 만들기
- ☒ 성장하는 서비스 만들기
- ☒ 우리도 함께 성장하기!

프로젝트 계획

- ☒ MVP 개발
- ☒ 베타 출시
- ☒ 사용자 확보
- ☒ 서비스 고도화

2 열심히 만들고 개선해도 좀처럼 유저가 잘 모이지 않았다.

열심히 만들었는데... 왜 이렇게 안 모이지? 뭐가 문제일까?

서비스 대시보드

가입자 수
128명
(지난 주 대비 +2)

DAU
23명
(지난 주 대비 -1)

기능 추가
버그 수정
성능 개선...

3 유저가 안 모이니까, 사이드가 오히려 더 해비해지고 PM의 압박도 심해졌다.

지금 속도로는 절대 안 돼요. 다음 달까지 가입자 1,000명은 넘겨야 투자 미팅할 수 있어요. 이제 개발보다 마케팅에 집중해야 해요!

개발도 뻥찬데... 마케팅까지? 이게 맞는 걸까?

PM 요구사항

- ☒ 유저 확보 최우선
- ☐ 마케팅 전략 수립
- ☐ 주간 지표 보고 필수
- ☐ 성과 없으면 계획 수정

4 유저 모으는 데만 집중하게 되면서, 원래 기대했던 성장과는 방향이 달라지고 스트레스가 커져 회사 일과 개인 시간까지 무너졌다.

내가 원했던 건 이게 아닌데... 계속 이대로 가는 게 맞을까?

광고 집행
콘텐츠 제작
커뮤니티 운영
지표 분석
제휴/이벤트

회사 업무 집중도 ↓
개인 시간 거의 없음
번아웃 위기

블로그 업데이트? SNS 광고 A/B 테스트

메시지: 내가 사이드 프로젝트를 통해 얻고자 하는 것을 명확하게 하고, 그것을 얻을 수 있는 환경인지를 잘 점검하자.

- ☒ 합류 전 목표와 기대를 명확히 하기
- ☒ 팀의 방향성과 나의 목표가 맞는지 확인하기
- ☒ 성장할 수 있는 환경인지 지속적으로 점검하기

핵심 요약

- **01 협업:** 협업 마인드를 장착하고 문서화하고 공유하는 습관을 갖자
- **02 실무 지식:** 네트워크, DB 등 — 미리 공부하여 기반을 쌓고 실무를 통해 폭발적으로 성장하자.
- **03 커리어:** 아닌 곳은 빠르게 판단하고 준비는 항상 하되, 한 번은 제대로 몰입하자.

Q & A

경청해 주셔서 감사합니다.